



ENGINEERING RESILIENCE

The Lifecycle of a Bulletproof C++ Object

Mastering Stack Unwinding, RAI, and noexcept for Robust Software.

fig 1.0



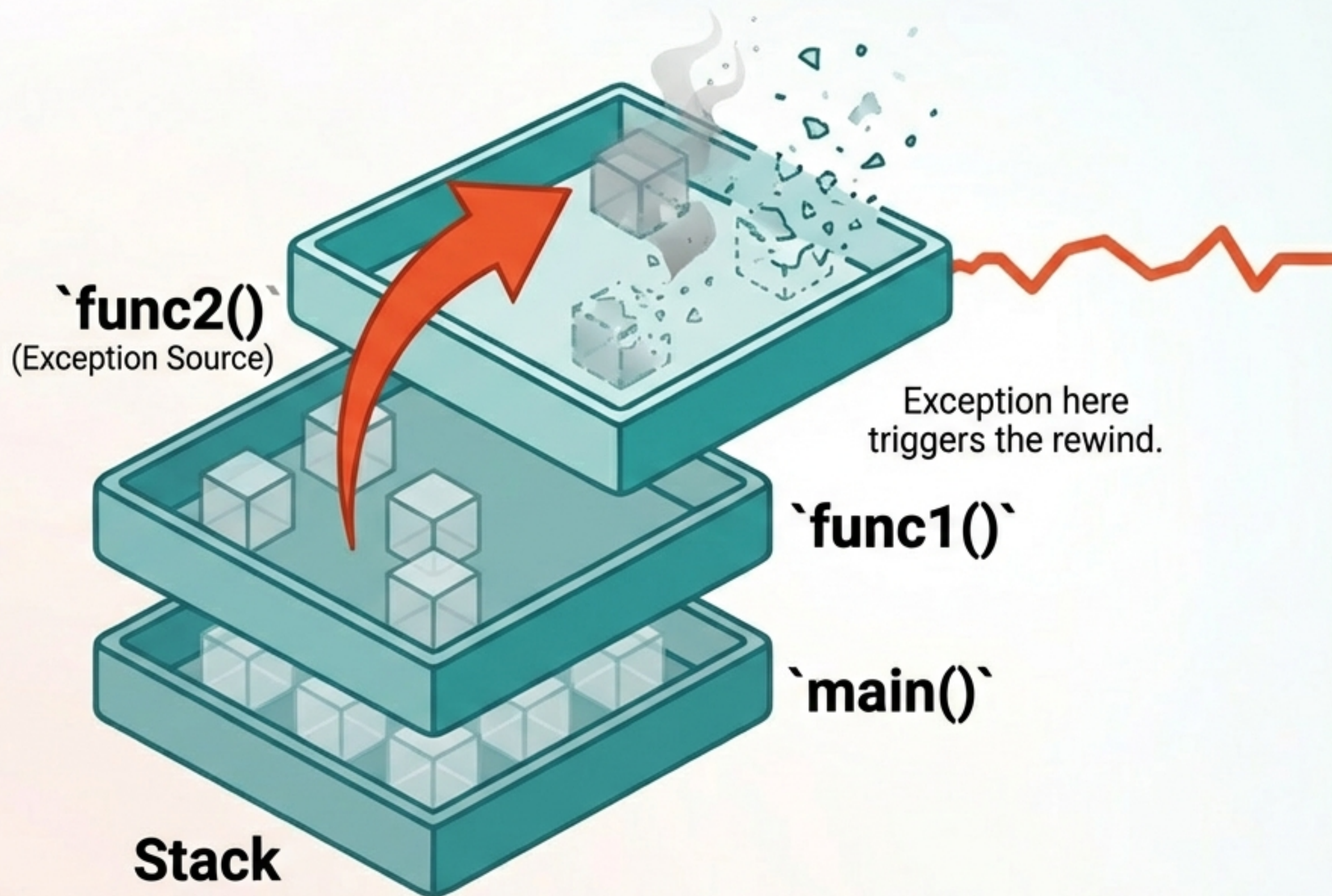
Exception Safety is More Than try and catch



True safety is about managing data existence in the face of chaos.
We must prevent catastrophic outcomes.

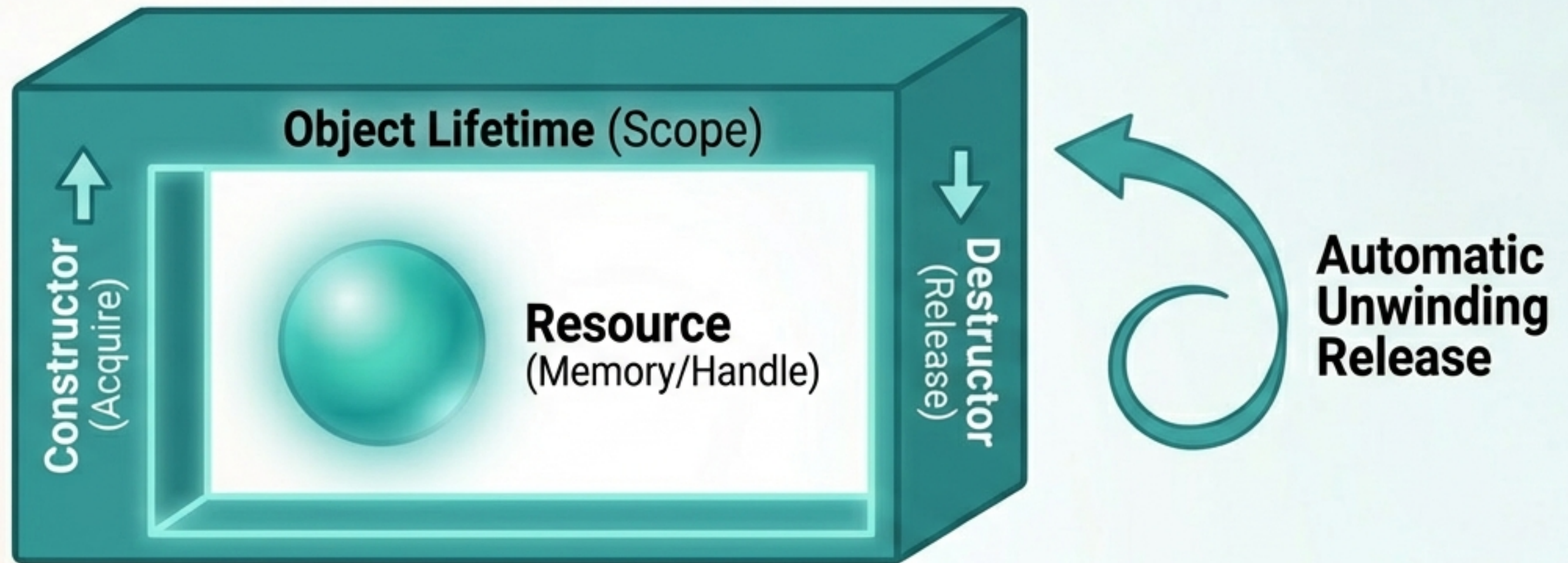
The Guarantee: Code must clean up after itself automatically, ensuring that valid state or no state remains.

The Mechanism: Visualizing Stack Unwinding



1. Functions exit in **reverse order**.
2. Local objects are destroyed automatically.
3. Destructors are **guaranteed** to execute.

The Golden Rule: Resource Acquisition Is Initialization (RAII)



Constructor

→ Acquire Resource



Destructor

→ Release Resource



Stack Unwinding

→ Trigger Destructor

By tying resources to scope, cleanup becomes deterministic.

The Cost of Manual Management

✗ Manual Management (Leak)

```
void bad() {  
    int* ptr = new int(10);  
    throw runtime_error("error");  
    delete ptr; // UNREACHABLE: Memory Leak  
}
```

Skipped during
unwinding



✓ RAI (Safe)

```
void good() {  
    unique_ptr<int> ptr = make_unique<int>(10);  
    throw runtime_error("error");  
} // Destructor runs automatically -> Memory freed
```



Cleanup Guaranteed



The Birth: Throwing from Constructors

Is it safe? **Yes.**

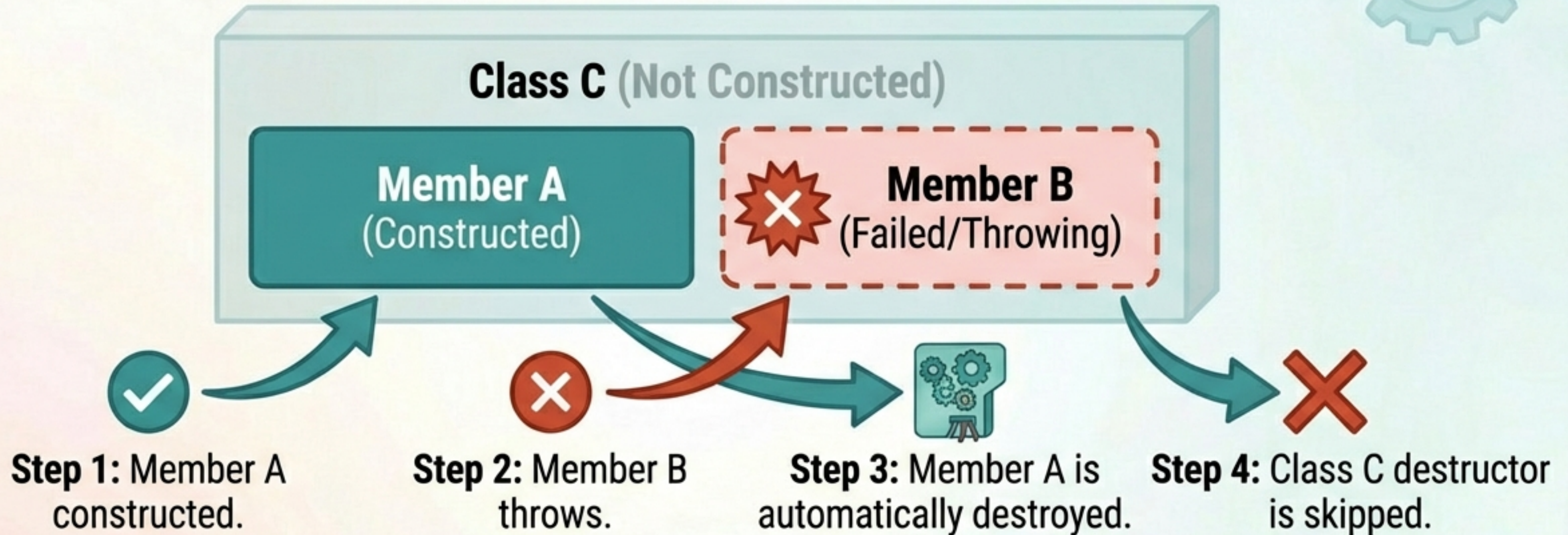


```
File(const char* filename) {  
    fp = fopen(filename, "r");  
    if (!fp) {  
        throw runtime_error("File open failed");  
        // Prevents invalid object creation  
    }  
}
```

Rationale: If an object cannot be properly initialized, it should not exist.

Handling Partial Construction

What happens if a constructor throws halfway through?



Result: No leaks, provided members use RAII.

The Death: The Danger of Destructors

⚠ **NEVER** Throw from a **Destructor**.

The Double Exception Collision

Existing Exception
(Unwinding)

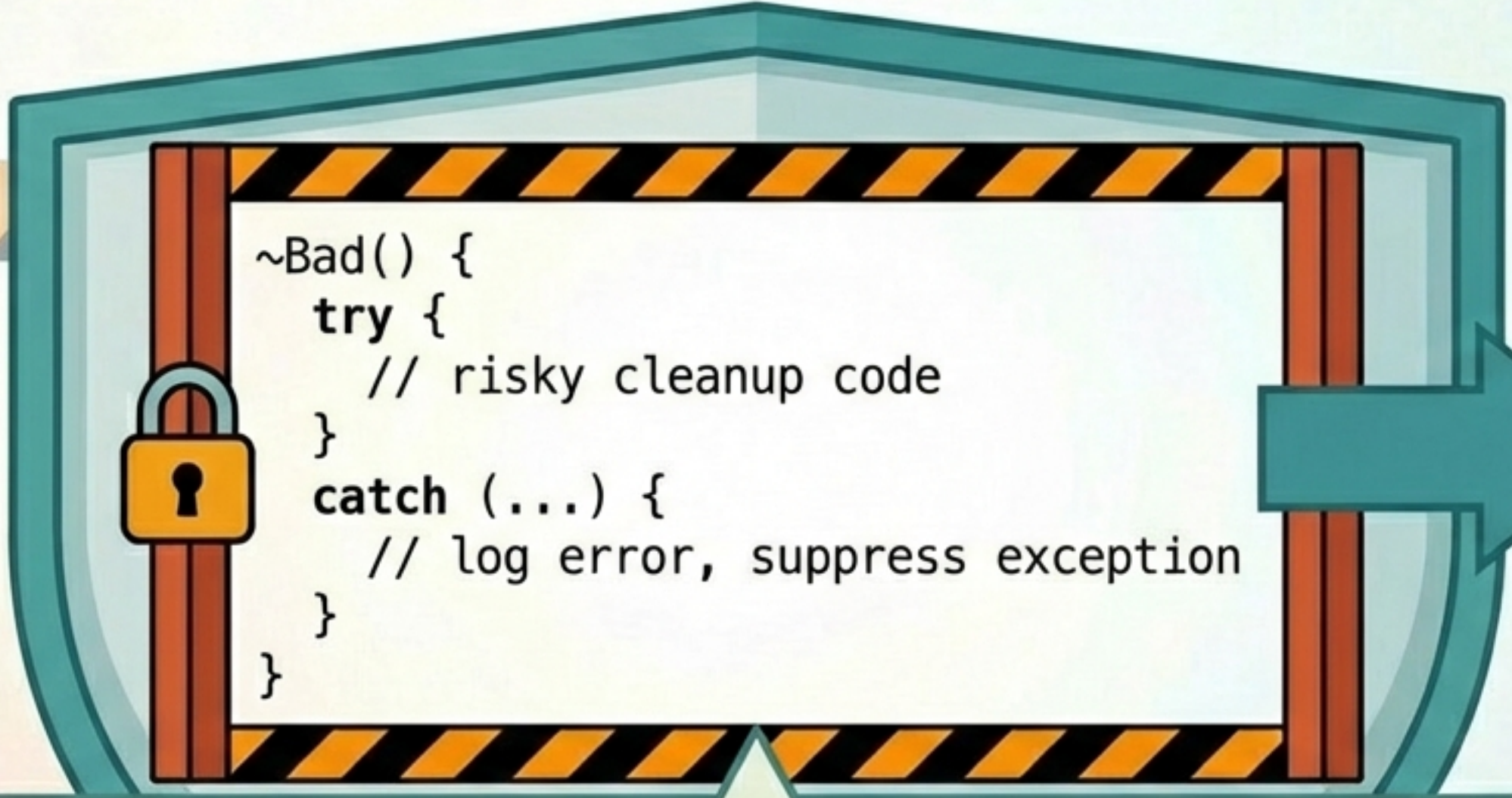
`std::terminate()`

New Exception
(from Destructor)

The C++ runtime **cannot** handle two active exceptions simultaneously. The result is an **immediate, unrecoverable crash**.

Safe Patterns for Destruction

Log it, swallow it, but never let it escape.

A diagram of a teal-colored safe with a yellow padlock on the left door handle. The safe is open, revealing a white interior with a yellow and black striped border at the top and bottom. Inside the safe, there is a code block. A large teal arrow points from the right side of the safe towards an 'Exception Log' icon, which is a document with a red 'X' over it. The background features several light blue gears and a teal pipe in the bottom right corner.

```
~Bad() {  
    try {  
        // risky cleanup code  
    }  
    catch (...) {  
        // log error, suppress exception  
    }  
}
```

In modern C++, destructors are implicitly **noexcept(true)**.
This identification with heat containment cannot be **loved**.

The Contract: Understanding noexcept

A specifier promising the compiler: “This function will not throw.”

```
void func() noexcept;
```



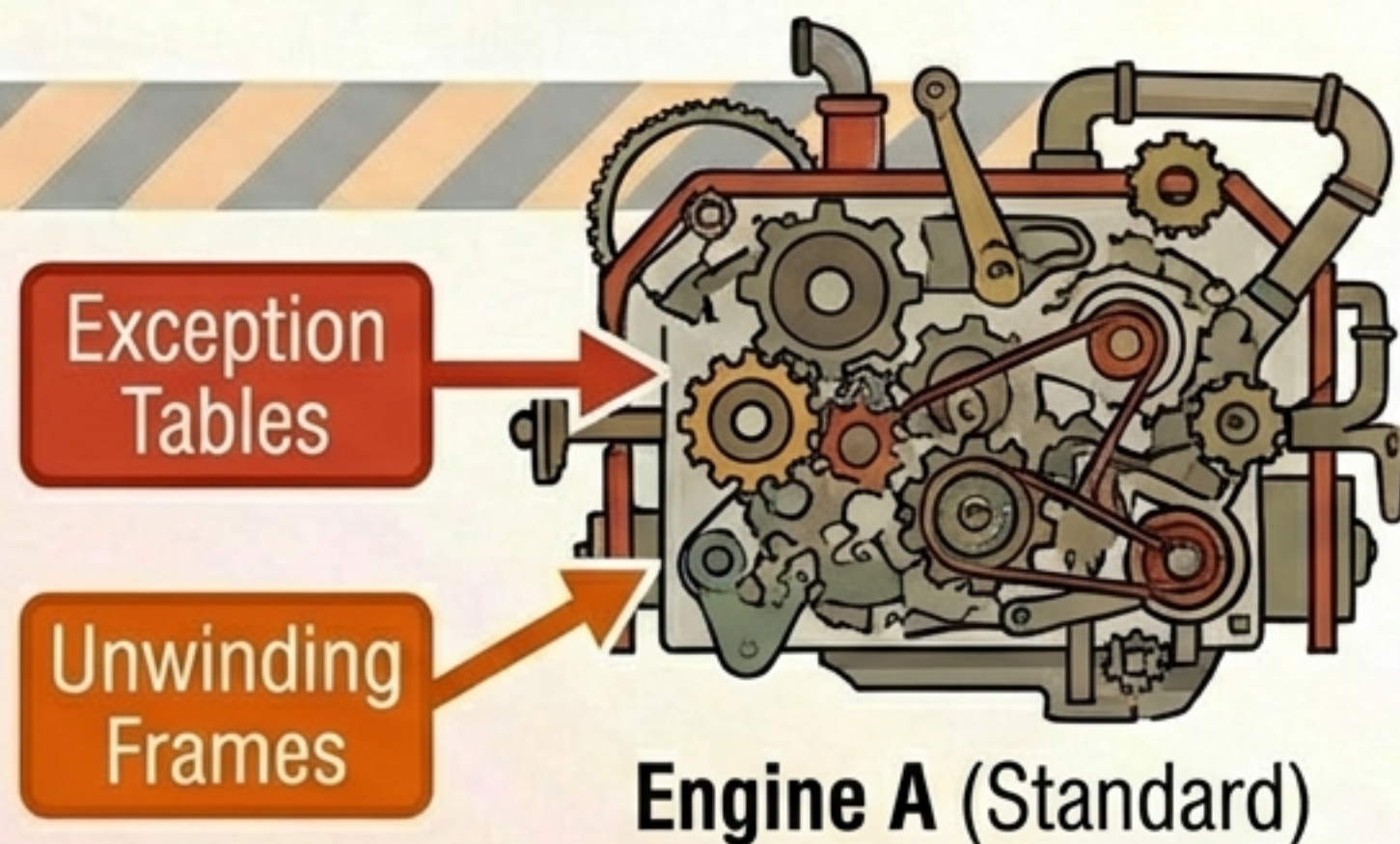
If promise is **kept**:
Optimized.

If promise is **broken**:
std::terminate().

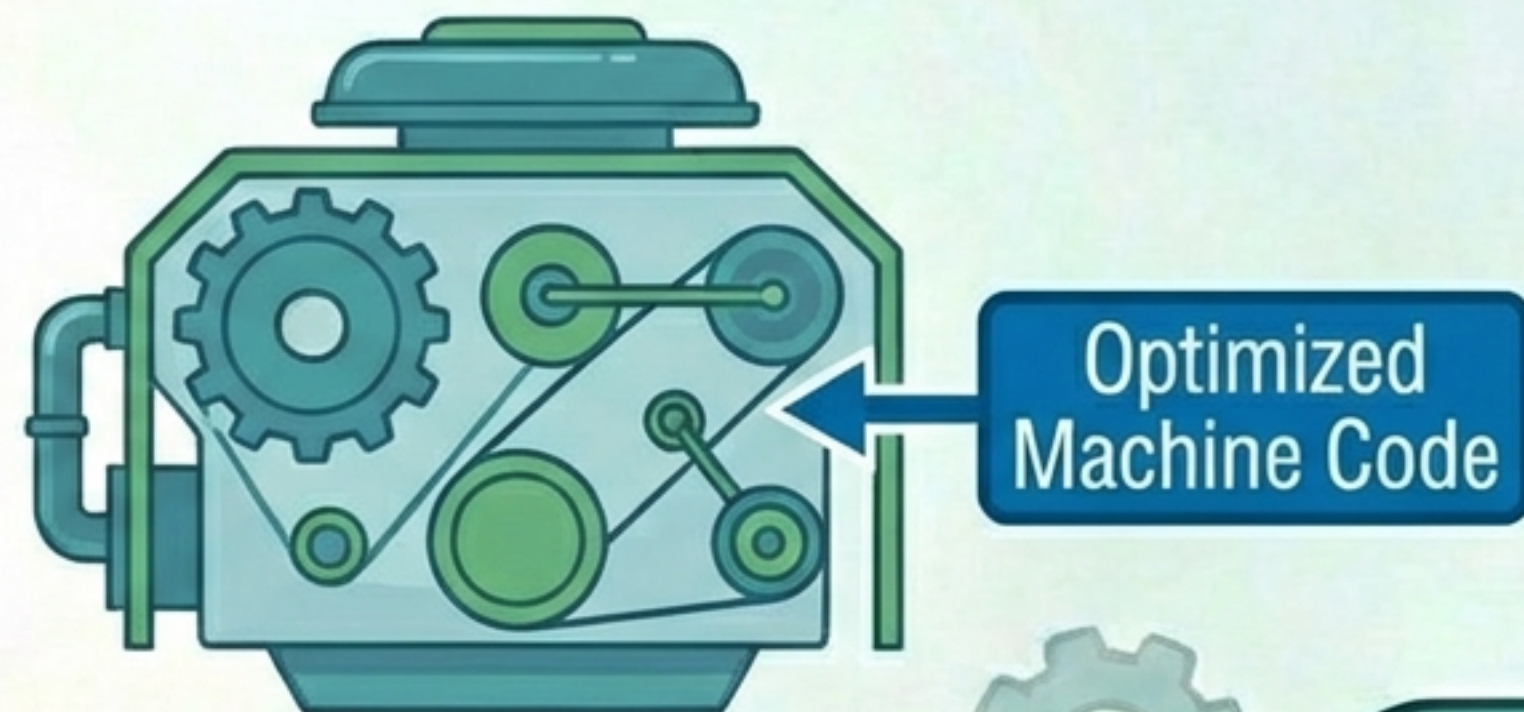
There is **no safety net**. Violating **noexcept** disables stack unwinding and crashes the program.

The Reward: Performance Optimization

When the compiler knows a function cannot throw, it removes exception-handling overhead and generates aggressive optimizations.



Engine A (Standard)



Engine B (noexcept)

`noexcept`(*expr*)

Can be used conditionally to query safety.

The STL Connection: Move Semantics

Scenario: Resizing a `std::vector`



The STL **prefers safety** over speed. If move operations aren't guaranteed **noexcept**, it falls back to copying to preserve data integrity.

The noexcept Cheat Sheet

Destructors	Always noexcept (Implicit Default)
Move Constructors	Highly Recommended (Enables STL Speed)
Swap Functions	Usually noexcept
General Functions	Only if 100% sure

If a function calls *anything* that might throw, it cannot be **noexcept**.

Summary: The Pillars of Resilience

Unwinding

Automatic destruction
in reverse order.

RAII

Resources tied to scope.
Prevents leaks.

Constructors

Safe to throw.
Prevents zombie objects.

Destructors

FATAL to throw.
Must catch internally.

Writing Robust C++

Valid State or No State.

- Lean on `unique_ptr` and `shared_ptr`.
- Don't fight the runtime.
- Design objects that align with the natural lifecycle.

```
std::vector<std::unique_ptr<T>> safe_data;
```